

**LAPORAN PRAKTIKUM**  
**STRUKTUR DATA**  
**“laporan Praktikum Pekan 9”**

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.  
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS ANDALAS

2026

## **KATA PENGANTAR**

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-9 mata kuliah Struktur Data yang membahas implementasi Binary Tree beserta operasi traversal (Preorder, Inorder, Postorder) dan representasi pohon secara visual, serta konsep dasar Graph Traversal menggunakan Depth-First Search (DFS) dan Breadth-First Search (BFS). Praktikum ini sangat penting untuk memahami struktur data non-linier yang banyak digunakan dalam berbagai aplikasi seperti pencarian, organisasi data, dan pemrosesan hierarki.

Kami menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi teknis maupun penyajian. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan laporan di masa yang akan datang.

**Padang, 11 Juni, 2026**

**Penulis**

## DAFTAR ISI

|                                       |     |
|---------------------------------------|-----|
| KATA PENGANTAR .....                  | ii  |
| DAFTAR ISI .....                      | iii |
| BAB I PENDAHULUAN .....               | 1   |
| 1.1 Latar Belakang .....              | 1   |
| 1.2 Tujuan .....                      | 1   |
| 1.3 Manfaat .....                     | 1   |
| BAB II PEMBAHASAN .....               | 2   |
| 2.1 Class Node .....                  | 2   |
| 2.1.1 Teori.....                      | 2   |
| 2.1.2 Langkah-langkah .....           | 2   |
| 2.1.3 Contoh Kode .....               | 2   |
| 2.1.4 Analisis .....                  | 2   |
| 2.2 Class BTree .....                 | 3   |
| 2.2.1 Teori .....                     | 3   |
| 2.2.2 Langkah-langkah .....           | 3   |
| 2.2.3 Contoh Kode .....               | 3   |
| 2.2.4 Analisis .....                  | 3   |
| 2.3 <b>Class GraphTraversal</b> ..... | 4   |
| 2.2.1 Teori .....                     | 4   |
| 2.2.2 Langkah-langkah .....           | 4   |
| 2.2.3 Contoh Kode .....               | 4   |

|                                     |           |
|-------------------------------------|-----------|
| 2.2.4 Analisis .....                | 4         |
| <b>2.4 Class Btree Driver .....</b> | <b>5</b>  |
| 2.2.1 Teori .....                   | 5         |
| 2.2.2 Langkah-langkah .....         | 5         |
| 2.2.3 Contoh Kode .....             | 5         |
| 2.2.4 Hasil .....                   | 5         |
| 2.2.5 Analisis .....                | 5         |
| <b>2.6 Tugas Pekan 9 .....</b>      | <b>6</b>  |
| 2.5.1. Analisis .....               | 6         |
| <b>BAB III KESIMPULAN .....</b>     | <b>9</b>  |
| <b>DAFTAR PUSTAKA .....</b>         | <b>10</b> |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Binary Tree merupakan salah satu struktur data non-linier yang paling fundamental dalam ilmu komputer. Setiap node pada Binary Tree memiliki maksimal dua anak (left dan right). Praktikum ini membahas implementasi Binary Tree secara manual beserta berbagai metode traversal serta konsep dasar traversal pada Graph menggunakan DFS dan BFS.

### **1.2 Tujuan**

1. Memahami konsep dan struktur Binary Tree.
2. Mengimplementasikan class Node dan Binary Tree beserta operasi traversal.
3. Memahami perbedaan Preorder, Inorder, dan Postorder traversal.
4. Mengimplementasikan Graph Traversal menggunakan DFS dan BFS.
5. Memahami penerapan Binary Tree dan Graph dalam bentuk program driver.

### **1.3 Manfaat**

1. Mahasiswa mampu mengimplementasikan struktur data pohon secara manual.
2. Memahami pentingnya traversal dalam eksplorasi data hierarki dan graf.
3. Memberikan fondasi yang kuat untuk mempelajari struktur data lanjutan seperti Binary Search Tree dan Graph Algorithm.

## BAB II

### PEMBAHASAN

#### 2.1 Class Node

##### 2.1.1 Teori

Node merupakan komponen dasar dari Binary Tree yang berisi data serta referensi ke node anak kiri (left) dan anak kanan (right).

##### 2.1.2 Langkah-langkah

1. Membuat class Node\_2511533026 dengan atribut data, left, dan right.
2. Membuat constructor dan method setter serta getter.
3. Menambahkan method untuk traversal.

##### 2.1.3 Contoh Kode

```
package Pemah_2511533026;

public class Node_2511533026 {
    int data;
    Node_2511533026 left;
    Node_2511533026 right;

    public Node_2511533026(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }

    public void setLeft(Node_2511533026 node) {
        if (left == null)
            left = node;
    }

    public void setRight(Node_2511533026 node) {
        if (right == null)
            right = node;
    }

    public Node_2511533026 getLeft() {
        return left;
    }

    public Node_2511533026 getRight() {
        return right;
    }

    public int getData() {
        return data;
    }

    public void setData(int data) {
        this.data = data;
    }

    void printPreorder(Node_2511533026 node) {
        if (node == null)
            return;
        System.out.print(node.data + " ");
        printPreorder(node.left);
        printPreorder(node.right);
    }

    void printInorder(Node_2511533026 node) {
        if (node == null)
            return;
        printInorder(node.left);
        System.out.print(node.data + " ");
        printInorder(node.right);
    }

    public String show() {
        return showPre() + showIn() + showPost();
    }

    public String showPre() {
        return showPre("Pre", node.data, showPre, showIn, showPost);
    }

    public String showIn() {
        return showIn("In", node.data, showPre, showIn, showPost);
    }

    public String showPost() {
        return showPost("Post", node.data, showPre, showIn, showPost);
    }
}
```

##### 2.1.4 Analisis

Class Node dirancang dengan baik menggunakan encapsulation. Method traversal ditempatkan di node untuk mendukung rekursi.

## 2.2 Class BTree

### 2.2.1 Teori

Binary Tree adalah struktur data di mana setiap node memiliki paling banyak dua anak. Class ini mengelola root dan menyediakan operasi traversal.

### 2.2.2 Langkah-langkah

1. Membuat root tree.
2. Mengimplementasikan method search, countNodes, dan ketiga jenis traversal.
3. Menyediakan method print tree dalam bentuk visual.

### 2.2.3 Contoh Kode

```
import java.util.*;

public class BTree {
    private Node root;
    private Node currentNode;
    public BTree() {
        root = null;
    }
    public boolean isEmpty() {
        return root == null;
    }
    public boolean insert(Node node, int data) {
        if (node == null) return false;
        if (data < node.data) {
            if (node.left == null) {
                node.left = new Node(data);
            } else {
                insert(node.left, data);
            }
        } else {
            if (node.right == null) {
                node.right = new Node(data);
            } else {
                insert(node.right, data);
            }
        }
        return true;
    }
    public void preorder() {
        preorder(root);
    }
    public void inorder() {
        inorder(root);
    }
    public void postorder() {
        postorder(root);
    }
    public Node getRoot() {
        return root;
    }
    public boolean isEmpty() {
        return root == null;
    }
}
```

```
import java.util.*;

public class BTree {
    private Node root;
    private Node currentNode;
    public BTree() {
        root = null;
    }
    public boolean isEmpty() {
        return root == null;
    }
    public boolean insert(Node node, int data) {
        if (node == null) return false;
        if (data < node.data) {
            if (node.left == null) {
                node.left = new Node(data);
            } else {
                insert(node.left, data);
            }
        } else {
            if (node.right == null) {
                node.right = new Node(data);
            } else {
                insert(node.right, data);
            }
        }
        return true;
    }
    public void preorder() {
        preorder(root);
    }
    public void inorder() {
        inorder(root);
    }
    public void postorder() {
        postorder(root);
    }
    public Node getRoot() {
        return root;
    }
    public boolean isEmpty() {
        return root == null;
    }
}
```

### 2.2.4 Analisis

Implementasi rekursif traversal sangat efisien. Method print() memberikan visualisasi pohon yang jelas.

## 2.3 Class GraphTraversal

### 2.3.1 Teori

Graph Traversal digunakan untuk mengunjungi semua node pada graf. Dua metode utama adalah DFS (Depth First Search) dan BFS (Breadth First Search).

### 2.3.2 Langkah-langkah

1. Representasi graf menggunakan Adjacency List (HashMap).
2. Implementasi DFS menggunakan rekursi.
3. Implementasi BFS menggunakan Queue.

### 2.3.3 Contoh Kode

```
// GraphTraversal.java
import java.util.*;

public class GraphTraversal {
    // Representasi graf menggunakan Adjacency List (HashMap)
    private Map<String, List<String>> graph;

    // Konstruktor
    public GraphTraversal() {
        graph = new HashMap<>();
    }

    // Menambahkan node ke graf
    public void addNode(String node) {
        graph.putIfAbsent(node, new ArrayList<>());
    }

    // Menambahkan sisi ke graf
    public void addEdge(String source, String target) {
        graph.get(source).add(target);
    }

    // Melakukan DFS
    public void dfs(String start) {
        dfsUtil(start, new ArrayList<>());
    }

    // Melakukan BFS
    public void bfs(String start) {
        bfsUtil(start, new ArrayList<>());
    }

    // DFS Utilitas
    private void dfsUtil(String current, List<String> visited) {
        if (!visited.contains(current)) {
            visited.add(current);
            for (String neighbor : graph.get(current)) {
                dfsUtil(neighbor, visited);
            }
        }
    }

    // BFS Utilitas
    private void bfsUtil(String start, List<String> visited) {
        Queue<String> queue = new LinkedList<>();
        queue.add(start);
        visited.add(start);
        while (!queue.isEmpty()) {
            String current = queue.poll();
            for (String neighbor : graph.get(current)) {
                if (!visited.contains(neighbor)) {
                    queue.add(neighbor);
                    visited.add(neighbor);
                }
            }
        }
    }

    // Main Method
    public static void main(String[] args) {
        GraphTraversal graph = new GraphTraversal();

        // Menambahkan node
        graph.addNode("A");
        graph.addNode("B");
        graph.addNode("C");
        graph.addNode("D");
        graph.addNode("E");

        // Menambahkan sisi
        graph.addEdge("A", "B");
        graph.addEdge("A", "C");
        graph.addEdge("B", "D");
        graph.addEdge("C", "E");

        // Melakukan DFS
        graph.dfs("A");

        // Melakukan BFS
        graph.bfs("A");
    }
}
```

### 2.3.4 Analisis

DFS cocok untuk eksplorasi mendalam, sedangkan BFS cocok untuk mencari jalur terpendek pada graf tidak berbobot.

## 2.4 Class BTreeDriver

### 2.4.1 Teori

Driver class digunakan untuk menguji seluruh fungsi Binary Tree yang telah dibuat.

### 2.4.2 Langkah-langkah

1. Membuat objek BTree.
2. Menambahkan node secara manual.
3. Melakukan berbagai traversal dan operasi count.

### 2.4.3 Contoh Kode

```
1 package prog01_2511533026;
2
3 public class BTreeDriver {
4     public static void main (String[] args) {
5         BTree tree = new BTree(1000000);
6         tree.add(1);
7         tree.add(2);
8         tree.add(3);
9         tree.add(4);
10        tree.add(5);
11        tree.add(6);
12        tree.add(7);
13        tree.add(8);
14        tree.add(9);
15        tree.add(10);
16        tree.add(11);
17        tree.add(12);
18        tree.add(13);
19        tree.add(14);
20        tree.add(15);
21        tree.add(16);
22        tree.add(17);
23        tree.add(18);
24        tree.add(19);
25        tree.add(20);
26        tree.add(21);
27        tree.add(22);
28        tree.add(23);
29        tree.add(24);
30        tree.add(25);
31        tree.add(26);
32        tree.add(27);
33        tree.add(28);
34        tree.add(29);
35        tree.add(30);
36        tree.add(31);
37        tree.add(32);
38        tree.add(33);
39        tree.add(34);
40        tree.add(35);
41        tree.add(36);
42        tree.add(37);
43        tree.add(38);
44        tree.add(39);
45        tree.add(40);
46        tree.add(41);
47        tree.add(42);
48        tree.add(43);
49        tree.add(44);
50        tree.add(45);
51        tree.add(46);
52        tree.add(47);
53        tree.add(48);
54        tree.add(49);
55        tree.add(50);
56        tree.add(51);
57        tree.add(52);
58        tree.add(53);
59        tree.add(54);
60        tree.add(55);
61        tree.add(56);
62        tree.add(57);
63        tree.add(58);
64        tree.add(59);
65        tree.add(60);
66        tree.add(61);
67        tree.add(62);
68        tree.add(63);
69        tree.add(64);
70        tree.add(65);
71        tree.add(66);
72        tree.add(67);
73        tree.add(68);
74        tree.add(69);
75        tree.add(70);
76        tree.add(71);
77        tree.add(72);
78        tree.add(73);
79        tree.add(74);
80        tree.add(75);
81        tree.add(76);
82        tree.add(77);
83        tree.add(78);
84        tree.add(79);
85        tree.add(80);
86        tree.add(81);
87        tree.add(82);
88        tree.add(83);
89        tree.add(84);
90        tree.add(85);
91        tree.add(86);
92        tree.add(87);
93        tree.add(88);
94        tree.add(89);
95        tree.add(90);
96        tree.add(91);
97        tree.add(92);
98        tree.add(93);
99        tree.add(94);
100       tree.add(95);
101       tree.add(96);
102       tree.add(97);
103       tree.add(98);
104       tree.add(99);
105       tree.add(100);
106       tree.add(101);
107       tree.add(102);
108       tree.add(103);
109       tree.add(104);
110       tree.add(105);
111       tree.add(106);
112       tree.add(107);
113       tree.add(108);
114       tree.add(109);
115       tree.add(110);
116       tree.add(111);
117       tree.add(112);
118       tree.add(113);
119       tree.add(114);
120       tree.add(115);
121       tree.add(116);
122       tree.add(117);
123       tree.add(118);
124       tree.add(119);
125       tree.add(120);
126       tree.add(121);
127       tree.add(122);
128       tree.add(123);
129       tree.add(124);
130       tree.add(125);
131       tree.add(126);
132       tree.add(127);
133       tree.add(128);
134       tree.add(129);
135       tree.add(130);
136       tree.add(131);
137       tree.add(132);
138       tree.add(133);
139       tree.add(134);
140       tree.add(135);
141       tree.add(136);
142       tree.add(137);
143       tree.add(138);
144       tree.add(139);
145       tree.add(140);
146       tree.add(141);
147       tree.add(142);
148       tree.add(143);
149       tree.add(144);
150       tree.add(145);
151       tree.add(146);
152       tree.add(147);
153       tree.add(148);
154       tree.add(149);
155       tree.add(150);
156       tree.add(151);
157       tree.add(152);
158       tree.add(153);
159       tree.add(154);
160       tree.add(155);
161       tree.add(156);
162       tree.add(157);
163       tree.add(158);
164       tree.add(159);
165       tree.add(160);
166       tree.add
```

## **2.6 Tugas Pekan 9**

### **2.6.1 Analisis Kode Program**

Program PetaKampus\_2511533026 merupakan aplikasi berbasis Graphical User Interface (GUI) yang mengimplementasikan pencarian jalur pada graf menggunakan algoritma BFS (Breadth-First Search) dan DFS (Depth-First Search). Program ini mensimulasikan peta kampus Universitas Andalas dengan node-node lokasi penting seperti Gerbang, Rektorat, Perpustakaan, dll.

#### **1. Struktur dan Arsitektur**

- Kelas ini mewarisi JFrame dan menggunakan BorderLayout untuk mengatur komponen antarmuka.
- Graf direpresentasikan menggunakan Adjacency List (Map<String, List<String>>) yang sangat efisien untuk graf tidak berbobot.
- Posisi visual node disimpan dalam HashMap<String, Point> untuk keperluan rendering.

#### **2. Fitur Utama**

- Visualisasi Graph: Menggunakan inner class GraphPanel yang meng-override paintComponent() untuk menggambar node, edge, dan path yang ditemukan dengan warna yang berbeda.
- Pencarian Jalur:
  - BFS: Menggunakan Queue untuk mencari jalur terpendek (shortest path) dalam jumlah edge.
  - DFS: Menggunakan Stack untuk eksplorasi mendalam.
- Path Reconstruction: Menggunakan HashMap sebagai parent map untuk merekonstruksi jalur dari start ke goal.
- Interaktivitas: ComboBox untuk memilih lokasi awal dan tujuan, tombol BFS, DFS, dan Reset.

#### **3. Teknik Pemrograman yang Digunakan**

- Event-Driven Programming dengan ActionListener untuk tombol.
- Custom Painting menggunakan Graphics2D dengan anti-aliasing untuk tampilan yang halus.
- State Management yang baik (visited set, path tracking, parent map).
- Thread Safety tidak digunakan secara eksplisit, namun operasi GUI dilakukan pada Event Dispatch Thread.

## Kelebihan Implementasi:

- Visualisasi graf dan path yang sangat jelas dan interaktif.
- Penggunaan warna berbeda untuk node yang dikunjungi, path hasil, dan edge.
- Rekonstruksi jalur yang benar menggunakan parent mapping.
- Desain UI yang user-friendly dengan label hasil pencarian yang informatif.

### Kekurangan / Saran Perbaikan:

- Belum ada penanganan kasus ketika tidak ada jalur (meskipun secara logika sudah ada).
- Algoritma DFS dan BFS bisa dijalankan pada thread terpisah agar UI tidak freeze saat graf sangat besar.
- Belum ada fitur animasi langkah demi langkah (hanya menampilkan hasil akhir).
- Koordinat node hardcoded (bisa dibuat lebih dinamis).

```
1 // Importing necessary libraries
2 import java.awt.*;
3 import java.awt.event.*;
4 import javax.swing.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7
8 public class GraphSearch {
9     // Graph representation
10     private Map<String, List<String>> adjList;
11     private Map<String, Integer> parentMap;
12     private List<String> visitedNodes;
13     private List<String> pathResult;
14     private List<String> startNodes;
15     private List<String> goalNodes;
16     private boolean isGoalReached;
17     private boolean isSearchComplete;
18
19     // Constructor
20     public GraphSearch() {
21         // Initialize adjacency list
22         adjList = new HashMap<>();
23         // Add nodes and edges
24         addNodes();
25         addEdges();
26     }
27
28     // Add nodes to the graph
29     private void addNodes() {
30         adjList.put("A", new ArrayList<>());
31         adjList.put("B", new ArrayList<>());
32         adjList.put("C", new ArrayList<>());
33         adjList.put("D", new ArrayList<>());
34         adjList.put("E", new ArrayList<>());
35         adjList.put("F", new ArrayList<>());
36         adjList.put("G", new ArrayList<>());
37         adjList.put("H", new ArrayList<>());
38         adjList.put("I", new ArrayList<>());
39         adjList.put("J", new ArrayList<>());
40     }
41
42     // Add edges to the graph
43     private void addEdges() {
44         adjList.get("A").add("B");
45         adjList.get("A").add("C");
46         adjList.get("B").add("A");
47         adjList.get("B").add("D");
48         adjList.get("C").add("A");
49         adjList.get("C").add("E");
50         adjList.get("D").add("B");
51         adjList.get("D").add("F");
52         adjList.get("E").add("C");
53         adjList.get("E").add("G");
54         adjList.get("F").add("D");
55         adjList.get("F").add("H");
56         adjList.get("G").add("E");
57         adjList.get("G").add("I");
58         adjList.get("H").add("F");
59         adjList.get("H").add("J");
60         adjList.get("I").add("G");
61         adjList.get("I").add("J");
62         adjList.get("J").add("H");
63         adjList.get("J").add("I");
64     }
65
66     // Main search method
67     public void search() {
68         // Initialize search parameters
69         visitedNodes = new ArrayList<>();
70         pathResult = new ArrayList<>();
71         startNodes = new ArrayList<>();
72         goalNodes = new ArrayList<>();
73         isGoalReached = false;
74         isSearchComplete = false;
75
76         // Start the search process
77         searchFromStartNodes();
78     }
79
80     // Search from start nodes
81     private void searchFromStartNodes() {
82         // Iterate over start nodes
83         for (String startNode : startNodes) {
84             // Perform DFS search
85             dfs(startNode);
86             // Check if goal is reached
87             if (isGoalReached) {
88                 // Display the result
89                 displayResult();
90                 return;
91             }
92         }
93     }
94
95     // Depth First Search (DFS)
96     private void dfs(String startNode) {
97         // Mark start node as visited
98         visitedNodes.add(startNode);
99         // Add start node to path
100         pathResult.add(startNode);
101
102         // Iterate over neighbors
103         for (String neighbor : adjList.get(startNode)) {
104             // Check if neighbor is not visited
105             if (!visitedNodes.contains(neighbor)) {
106                 // Recursively call dfs
107                 dfs(neighbor);
108                 // Check if goal is reached
109                 if (isGoalReached) {
110                     // Display
```

```

200     return result;
201 }
202 void Graph::clear() {
203     pathFound.clear();
204     pathFound.clear();
205     pathFound.clear();
206     displayPath();
207 }
208 void Graph::setPath(string path) {
209     pathFound = path;
210     pathFound.clear();
211     pathFound.clear();
212     pathFound.clear();
213     displayPath();
214 }
215 void Graph::setPath(string path) {
216     pathFound = path;
217     pathFound.clear();
218     pathFound.clear();
219     pathFound.clear();
220     displayPath();
221 }
222 void Graph::setPath(string path) {
223     pathFound = path;
224     pathFound.clear();
225     pathFound.clear();
226     pathFound.clear();
227     displayPath();
228 }
229 void Graph::setPath(string path) {
230     pathFound = path;
231     pathFound.clear();
232     pathFound.clear();
233     pathFound.clear();
234     displayPath();
235 }
236 void Graph::setPath(string path) {
237     pathFound = path;
238     pathFound.clear();
239     pathFound.clear();
240     pathFound.clear();
241     displayPath();
242 }
243 void Graph::setPath(string path) {
244     pathFound = path;
245     pathFound.clear();
246     pathFound.clear();
247     pathFound.clear();
248     displayPath();
249 }
250 void Graph::setPath(string path) {
251     pathFound = path;
252     pathFound.clear();
253     pathFound.clear();
254     pathFound.clear();
255     displayPath();
256 }
257 void Graph::setPath(string path) {
258     pathFound = path;
259     pathFound.clear();
260     pathFound.clear();
261     pathFound.clear();
262     displayPath();
263 }
264 void Graph::setPath(string path) {
265     pathFound = path;
266     pathFound.clear();
267     pathFound.clear();
268     pathFound.clear();
269     displayPath();
270 }
271 void Graph::setPath(string path) {
272     pathFound = path;
273     pathFound.clear();
274     pathFound.clear();
275     pathFound.clear();
276     displayPath();
277 }
278 void Graph::setPath(string path) {
279     pathFound = path;
280     pathFound.clear();
281     pathFound.clear();
282     pathFound.clear();
283     displayPath();
284 }
285 void Graph::setPath(string path) {
286     pathFound = path;
287     pathFound.clear();
288     pathFound.clear();
289     pathFound.clear();
290     displayPath();
291 }
292 void Graph::setPath(string path) {
293     pathFound = path;
294     pathFound.clear();
295     pathFound.clear();
296     pathFound.clear();
297     displayPath();
298 }
299 void Graph::setPath(string path) {
300     pathFound = path;
301     pathFound.clear();
302     pathFound.clear();
303     pathFound.clear();
304     displayPath();
305 }
306 void Graph::setPath(string path) {
307     pathFound = path;
308     pathFound.clear();
309     pathFound.clear();
310     pathFound.clear();
311     displayPath();
312 }
313 void Graph::setPath(string path) {
314     pathFound = path;
315     pathFound.clear();
316     pathFound.clear();
317     pathFound.clear();
318     displayPath();
319 }
320 void Graph::setPath(string path) {
321     pathFound = path;
322     pathFound.clear();
323     pathFound.clear();
324     pathFound.clear();
325     displayPath();
326 }
327 void Graph::setPath(string path) {
328     pathFound = path;
329     pathFound.clear();
330     pathFound.clear();
331     pathFound.clear();
332     displayPath();
333 }
334 void Graph::setPath(string path) {
335     pathFound = path;
336     pathFound.clear();
337     pathFound.clear();
338     pathFound.clear();
339     displayPath();
340 }
341 void Graph::setPath(string path) {
342     pathFound = path;
343     pathFound.clear();
344     pathFound.clear();
345     pathFound.clear();
346     displayPath();
347 }
348 void Graph::setPath(string path) {
349     pathFound = path;
350     pathFound.clear();
351     pathFound.clear();
352     pathFound.clear();
353     displayPath();
354 }
355 void Graph::setPath(string path) {
356     pathFound = path;
357     pathFound.clear();
358     pathFound.clear();
359     pathFound.clear();
360     displayPath();
361 }
362 void Graph::setPath(string path) {
363     pathFound = path;
364     pathFound.clear();
365     pathFound.clear();
366     pathFound.clear();
367     displayPath();
368 }
369 void Graph::setPath(string path) {
370     pathFound = path;
371     pathFound.clear();
372     pathFound.clear();
373     pathFound.clear();
374     displayPath();
375 }
376 void Graph::setPath(string path) {
377     pathFound = path;
378     pathFound.clear();
379     pathFound.clear();
380     pathFound.clear();
381     displayPath();
382 }
383 void Graph::setPath(string path) {
384     pathFound = path;
385     pathFound.clear();
386     pathFound.clear();
387     pathFound.clear();
388     displayPath();
389 }
390 void Graph::setPath(string path) {
391     pathFound = path;
392     pathFound.clear();
393     pathFound.clear();
394     pathFound.clear();
395     displayPath();
396 }
397 void Graph::setPath(string path) {
398     pathFound = path;
399     pathFound.clear();
400     pathFound.clear();
401     pathFound.clear();
402     displayPath();
403 }
404 void Graph::setPath(string path) {
405     pathFound = path;
406     pathFound.clear();
407     pathFound.clear();
408     pathFound.clear();
409     displayPath();
410 }
411 void Graph::setPath(string path) {
```

## **BAB III**

### **KESIMPULAN**

Praktikum pekan ke-9 ini telah memberikan pemahaman yang mendalam mengenai struktur data **Binary Tree** beserta berbagai metode traversal serta konsep dasar **Graph Traversal** menggunakan DFS dan BFS. Melalui implementasi class Node, BTree, GraphTraversal, dan Driver, kami berhasil memahami cara kerja rekursif traversal dan representasi graf menggunakan adjacency list.

Praktikum ini sangat bermanfaat sebagai fondasi untuk mempelajari struktur data yang lebih kompleks seperti Binary Search Tree, AVL Tree, dan algoritma graph yang lebih advance. Pengetahuan yang diperoleh akan sangat membantu dalam menyelesaikan berbagai permasalahan yang melibatkan data hierarki dan hubungan antar elemen.

## DAFTAR PUSTAKA

1. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson.
2. Cormen, T. H., et al. (2009). *Introduction to Algorithms*. MIT Press.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
4. Materi Kuliah Struktur Data – Universitas Andalas.
5. Dokumentasi Java Collections Framework – Oracle.